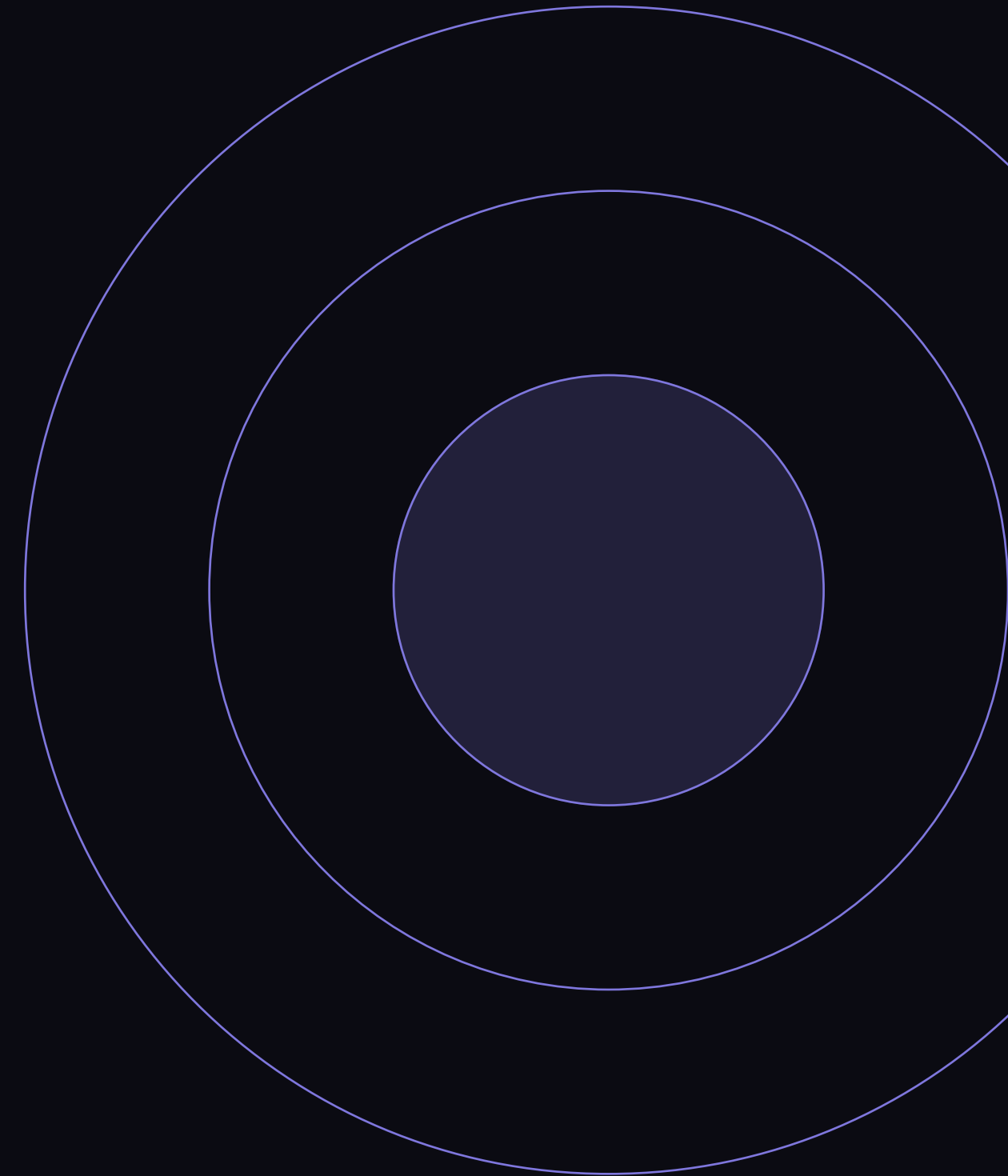


G-OOP-400 · EPITECH · C++

Raytracer

Moteur de rendu 3D photoréaliste
par simulation inverse de la lumière



Comment simuler la lumière ?

La lumière réelle rebondit, se réfléchit, projette des ombres.

La reproduire numériquement pixel par pixel est un problème mathématique complexe.

Le rendu classique sacrifie le réalisme. Notre objectif : le retrouver.

- **Raycasting** — 1 rayon $\rightarrow$ 1 impact $\rightarrow$ 1 pixel. Rapide, irréaliste
- **Raytracing** — Lancer récursif, la lumière rebondit vraiment
- **Le défi** — Simuler l'inverse du chemin de la lumière pour chaque pixel
- **Contrainte** — En C++ pur, sans moteur tiers, architecture extensible dès le départ

Un moteur modulaire, extensible, plugins-first

Cœur indépendant

Le moteur ne connaît aucune géométrie concrète. Tout passe par des interfaces abstraites — IPrimitive, ILight, IMateriau

Plugins dynamiques (.so)

Chaque primitive est une librairie dynamique chargée via dlopen/dlsym à l'exécution. Zéro recompilation du moteur

Design patterns

Factory pour construire les scènes depuis un fichier .cfg. Observer pour les événements de rendu. Architecture testable et maintenable

Scène déclarative

Caméra, primitives, lumières — tout configuré en libconfig++. Rien de hardcodé dans le binaire

Du fichier scène au pixel final

Pour chaque pixel, on lance un rayon depuis la caméra. Selon ce qu'il touche, on calcule couleur, ombre et réflexion récursivement.

1

Lecture du .cfg

Scène instanciée via Factory : caméra, primitives, lumières

2

Chargement des plugins

dlopen() charge chaque primitive .so au runtime

3

Lancer de rayons

Pour chaque pixel — test d'intersection avec toutes les primitives

4

Calcul lumière

Ombres portées, lumière ambiante/directionnelle, réflexions

5

Output PPM

Image écrite pixel par pixel en format PPM standard

Contrats stricts, couplage minimal

Chaque plugin implémente IPrimitive et expose un seul point d'entrée C. Le moteur ne voit que l'abstraction.

```
// plane.cpp - exemple réel
extern "C" IPrimitive *myEntryPoint()
{

    return new Plane();
}
```

PLUGINS CHARGÉS DYNAMIQUEMENT

sphere.so

plane.so

cone.so

cylinder.so

triangle.so

torus.so

mobius.so

tanglecube.so

fractal.so

objmesh.so

- Aucun plugin lié statiquement au binaire
- Ajouter une primitive = créer un .so, rien d'autre

10 primitives — du basique au géométriquement complexe

MUST / SHOULD ✓

- **Sphère, Plan** — Primitives fondamentales
- **Cylindre, Cône** — Formes limitées et infinies
- **Triangle** — Via algorithme de Möller-Trumbore
- **OBJ Mesh** — Import de fichiers .obj

COULD (BONUS) ★

- **Tore** — Surface de révolution quadrique
- **TangleCube** — Surface implicite algébrique
- **Fractal** — Ensemble de Julia/Mandelbrot
- **Ruban de Möbius** — Surface non-orientable

01

Multithreading

Rendu parallèle par blocs. Gain $\times 4$
estimé sur 4 cœurs — de 0.3s à ~ 0.08 s
sur scène basique

02

Modèle de Phong

Ambiant + diffus + spéculaire.
Matériaux bien plus réalistes, brillance
paramétrable

03

Interface graphique

Interface de rendu

L'architecture plugin permet d'ajouter ces features sans toucher au moteur — c'est exactement pour ça qu'on l'a conçue ainsi.

Équipe de 4 — répartition des responsabilités

Moteur core & interfaces (Ange-Marie)

Pipeline de rendu principal, système de plugins, gestion des interfaces IPrimitive / ILight / IMateriau

Mathématiques & transformations (Emmanuel)

Bibliothèque Math:: partagée — vecteurs, matrices, rotations, translations

Primitives avancées (Helena)

Plan infini, Triangle (Möller-Trumbore), Ruban de Möbius, rotations matricielles

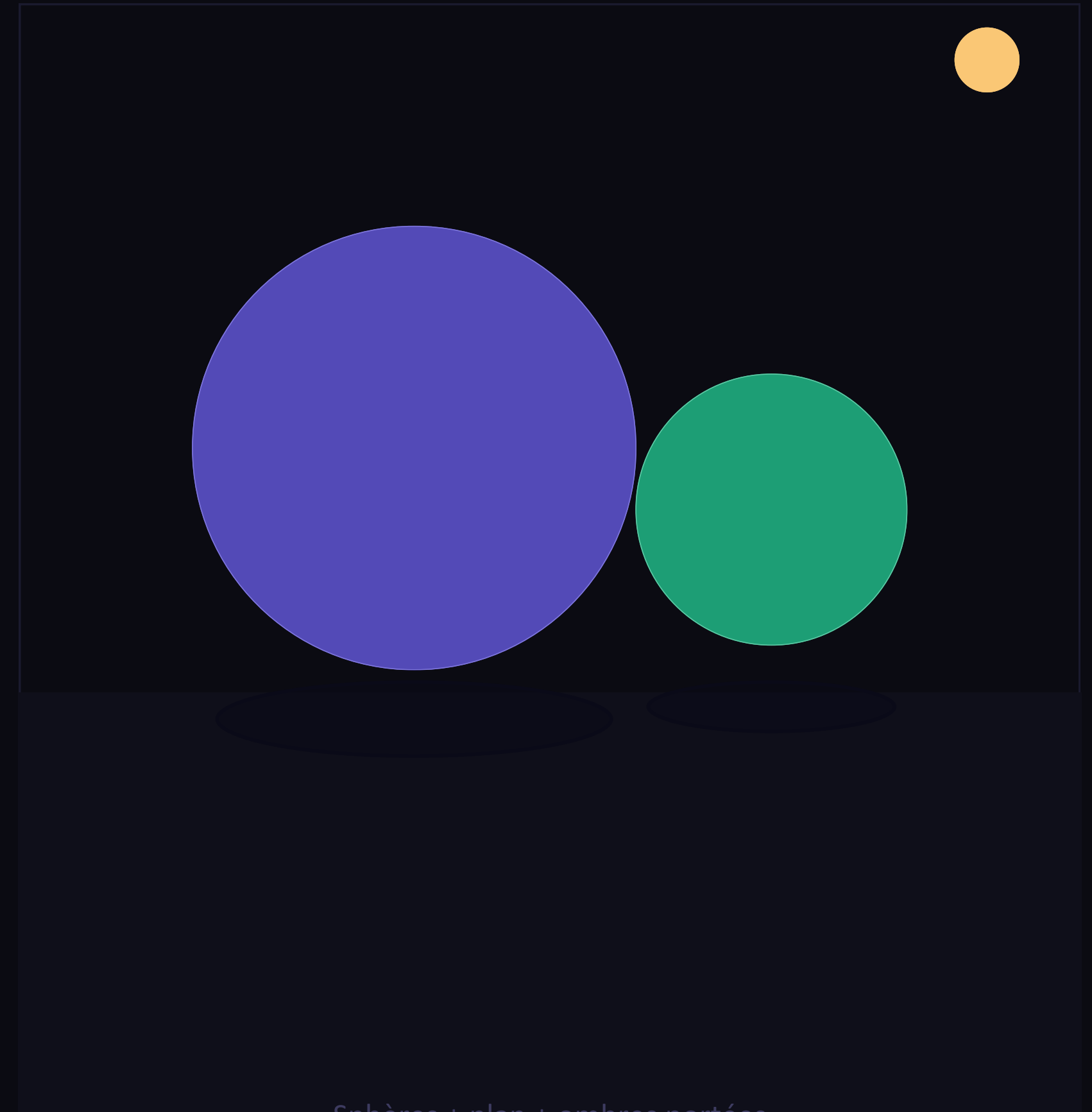
Scènes & configuration (Aquilas)

35 fichiers .cfg, parser libconfig++, Factory Pattern, lumières multiples

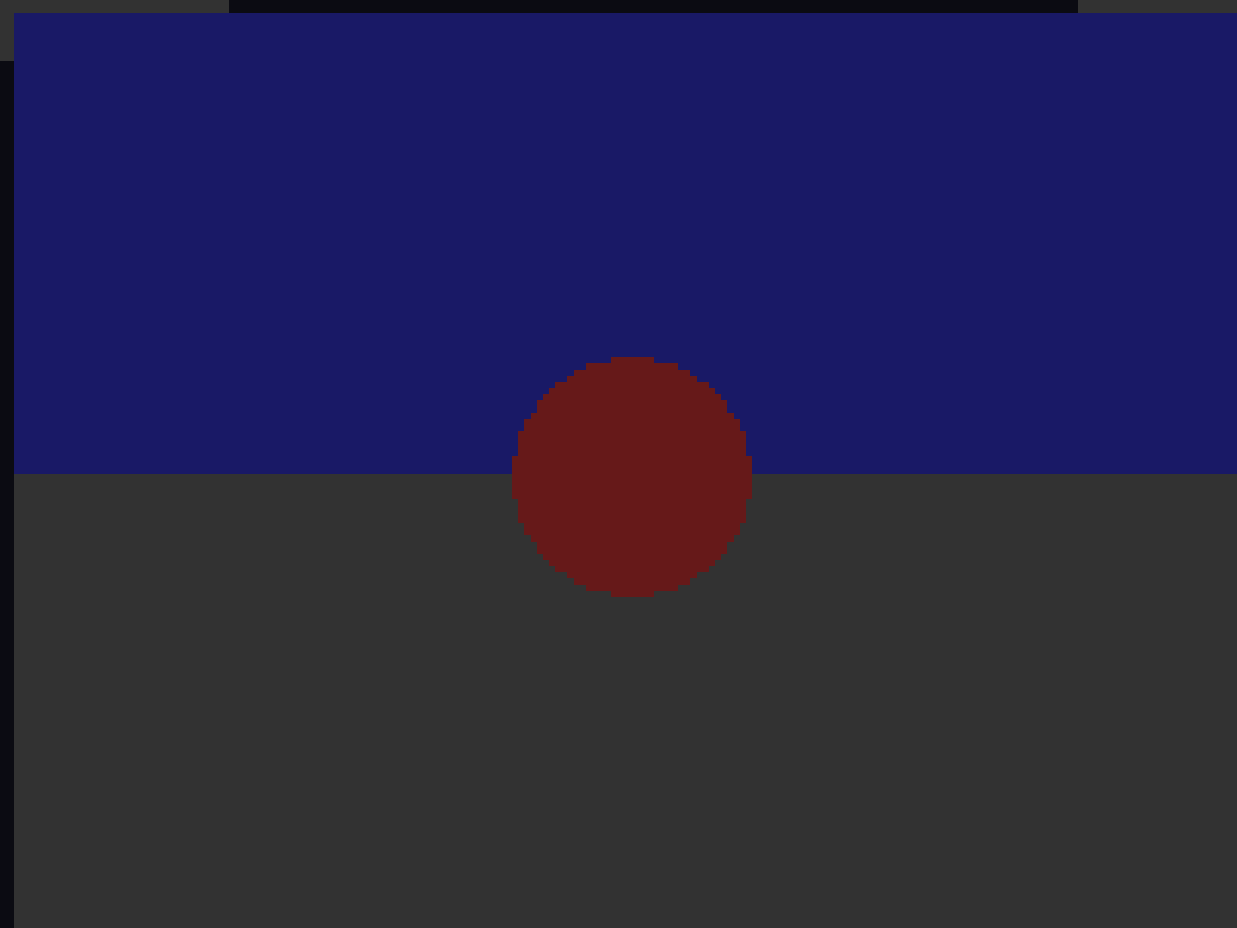
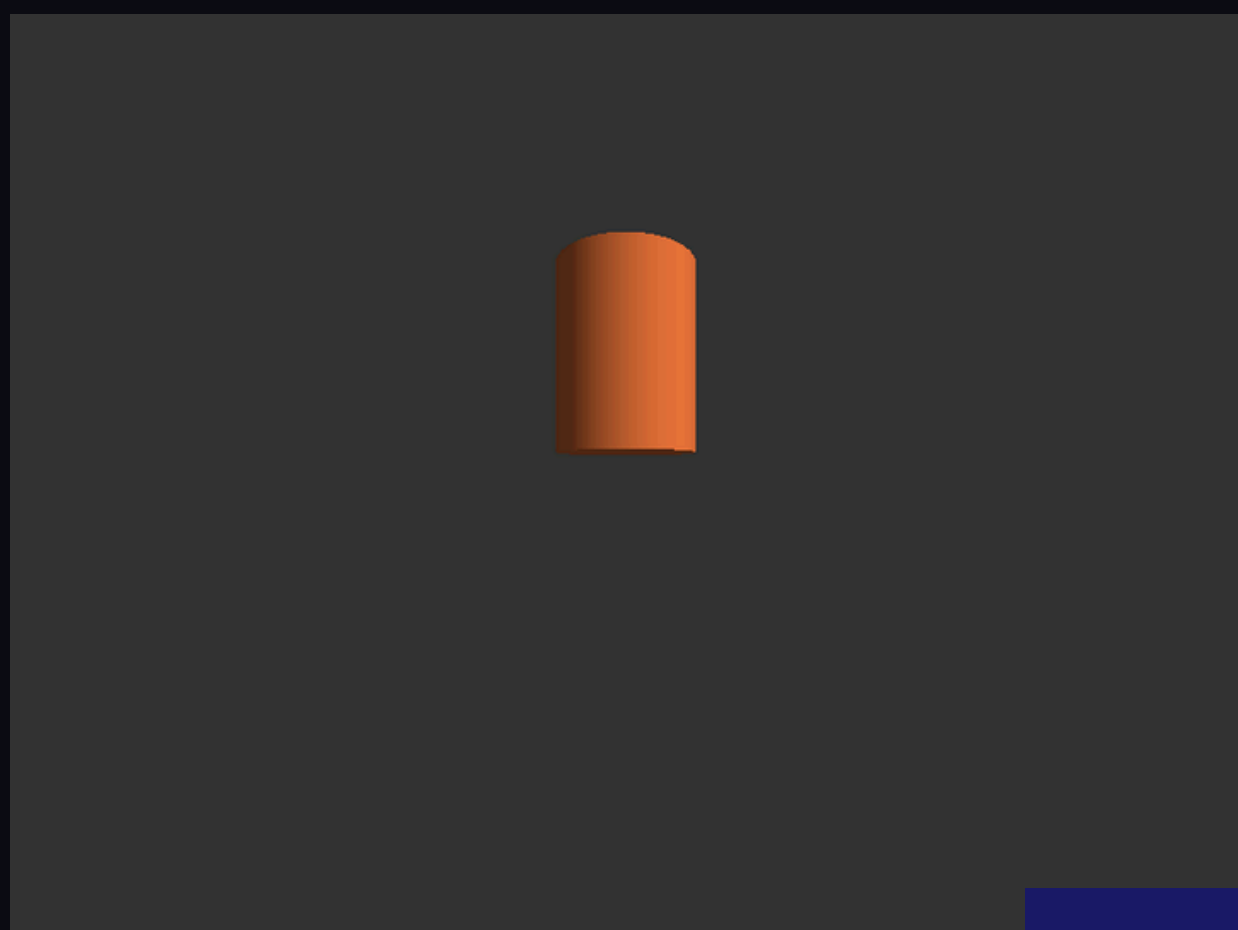
Scènes produites par le moteur

- Ombres portées — lumière directionnelle calculée
- Réflexions récursives sur les sphères
- Scènes complexes : tore + cylindre + plan + lumières multiples
- Output PPM, résolution 1920×1080 supportée

35 scènes pré-rendues disponibles. Démo live prête — aucune attente pendant la soutenance.



Sphères + plan + ombres portées



Animation 3d

Jeux vidéos

CONCLUSION

Un moteur qui va au-delà du minimal.

Architecture modulaire réelle, 10 primitives fonctionnelles, 35 scènes testées. Le projet prouve qu'on a compris les enjeux — pas juste fait tourner des sphères.

Must ✓

Should ✓

4 bonus Could

Plugins .so

Design patterns

82 fichiers